Governing AI in Software Engineering: The Case for Spec-Driven Development

---

Every serious organisation operating software systems rests on one assumption: that you can account for what your systems do. Not in general terms — specifically. Which rule does this component enforce? Which control does this process satisfy? When this behaviour changes, what changes with it?

That assumption is under pressure. AI is writing significant portions of enterprise software today, and the pace is increasing. The tooling is capable. The governance is not keeping up. The result is a growing inventory of production systems where the chain from business requirement to delivered behaviour is informal, undocumented, and unverifiable — exactly the condition that any serious governance framework is designed to prevent.

Spec-Driven Development is the engineering discipline that restores accountability. It is not a process layer on top of AI development. It is the architecture that makes AI development auditable by construction.

A Problem We Have Seen Before

Two decades ago, spreadsheets became the productivity tool that every business adopted enthusiastically. They were fast, flexible, required no IT involvement, and delivered immediate value. Every department built on them.

What followed was twenty years of governance reckoning. Business-critical calculations lived in files on personal drives. Pricing logic sat in models that only one person understood. Risk computations were embedded in spreadsheets that no system of record contained. When auditors asked how a number was arrived at, the answer was often a file called FINAL_v3_REVISED.xlsx, last modified by someone who had since left.

Organisations spent enormous effort — governance frameworks, data warehouses, validation processes, change management — trying to regain visibility over what their own systems were computing.

AI-generated code is that problem, running at an exponential scale.

A developer can now produce in an afternoon what previously took a team a week. That capability is already in use across the enterprise. The code it produces is in production systems, running business logic, making decisions. And unless something specific was done to prevent it, nobody — including the developer — can fully account for what it does, which business requirement it satisfies, or what changes when something goes wrong.

The spreadsheet problem was contained by what a spreadsheet could do. AI has no such limit. It writes entire systems.

Spec-Driven Development is the governance framework built for this moment. It does not slow AI down. It gives organisations the full productivity of AI construction with complete visibility into what is being built, why, and whether it matches what the business asked for. The spreadsheet era ended in governance debt. This one does not have to.

The Accountability Gap

Consider how a critical business system is built today under AI-assisted development. A developer describes what they need, the AI produces code, the developer reviews it, and the cycle continues until the output looks right. This is faster than writing the code manually. It produces the same accountability problem that has always existed in software development, now at higher velocity.

The accountability problem is this: the chain from business requirement to live behaviour is a series of informal handoffs. The requirement is stated. Something is built. The developer believes it matches. There is no structural mechanism that verifies the chain is intact — or that detects when it breaks.

For any organisation that relies on software to run its operations, that gap is not a quality issue. It is a risk issue. An undiscovered deviation between specified behaviour and actual behaviour in a critical system is not a bug to be fixed in the next sprint. It is a control failure. It requires investigation, remediation, and an audit trail demonstrating what went wrong and how it was corrected. The cost is measured in operational disruption and governance failures, not user complaints.

Spec-Driven Development applies the logic of formal governance to the software delivery process itself.

What Governed Delivery Looks Like

The methodology establishes formal governance at every stage of delivery: business intent, requirements, feature decomposition, architecture, code, testing, deployment, and production monitoring.

At each stage, the work produces a deliverable. Before that deliverable is accepted and the next stage begins, a defined set of checks must pass. Some are automated — does the code satisfy the test suite, does the coverage report confirm every requirement has been addressed. Some use AI to evaluate quality — does the architecture honour the constraints that were specified, does the implementation match the design intent. Some require human approval — is the feature decomposition complete, has the architecture been reviewed and accepted.

These are not process steps that teams are asked to follow. They are gates. The system will not advance without them. There is no path around them. The result is a delivery record that reflects what actually happened, not what the team intended to happen.

This is the same principle that governs any serious quality or compliance framework: independent verification, documented methodology, ongoing monitoring, and a record that can withstand examination. Applied to software delivery.

Traceability: From Requirement to Production

Every business requirement in the system is tagged with a unique identifier at the point of specification. That tag travels with the requirement through every downstream stage.

When the team decomposes requirements into features, an automated check confirms that every requirement tag is covered. If any requirement is unaddressed, the check fails and delivery does not advance. There is no way to miss a requirement silently.

The same tags appear in the architecture, the code, the test cases, and the production monitoring configuration. At any point — including during an audit or governance examination — you can answer: which test validates this requirement? Which production control is watching it? Which code implements it? Which change introduced it, and when?

This is what complete traceability looks like: not a document that describes the system, but a live record embedded in the delivery process. The document describes what was intended. The record proves what was done.

Production: Continuous Verification, Not Post-Hoc Detection

For any organisation running critical software, the most dangerous failure mode is not a system that crashes. It is a system that drifts — behaving slightly differently from its specification in ways that are not immediately visible, accumulating deviation until something material surfaces.

Spec-Driven Development closes the loop at production. The live system is monitored using the same requirement tags that governed its construction. When behaviour deviates from the specification — error rates rise, latency profiles shift, outputs fall outside expected ranges — the system identifies which requirement is in drift and initiates a formal response.

That response follows the same governed delivery process that built the system originally. It is specified, reviewed against dependent requirements, built, tested, and deployed with a full audit trail. There are no emergency patches outside the process. The process is the same process, triggered by a production signal instead of a business request.

This matters for operational resilience. The system does not wait for a user complaint or a risk report to detect that something has changed. It detects it, characterises it against the specification, and starts the correction. The audit trail from production anomaly to remediated deployment is complete and continuous.

Risk Management for AI Development Itself

Consider how organisations treat a new hire — even a brilliant graduate from a top institution. Capable. Intelligent. Full of energy. You would not hand them the keys to the business and say "go for it." You would give them context, constraints, and defined expectations. You would review their work. You would not promote them to senior leadership until they had demonstrated judgment in your specific environment — because raw intelligence is not the same as knowing your business.

An LLM is a brilliant graduate who has read everything ever written, knows every programming language, and has zero knowledge of your organisation. It will answer every question with confidence. It will never say "I don't know your constraints." It will produce something that looks exactly right, even when it isn't — because it has no way to know the difference.

Prompt-driven development is handing that graduate the keys and asking them to go for it. There is a specific risk here that conventional frameworks have not yet fully addressed: the AI has no knowledge of your requirements. It is capable, fast, and entirely unaware of what your business actually needs.

In prompt-driven development, the developer's skill at articulating requirements to the model determines what gets built. That is not a repeatable, auditable, or scalable governance mechanism. It is a key-person dependency dressed up as productivity tooling.

Spec-Driven Development separates the specification — what the system must do — from the construction — how it is built. The specification is the asset. It is formal, versioned, requirement-tagged, and independent of any particular AI model or developer. The AI models do the construction work against that specification and are evaluated by whether their output satisfies it. The specification does not change because the model misunderstood something. The model iterates until it satisfies the specification.

This is the correct inversion of control. The business requirement governs the AI. Not the other way around.

What This Is and Is Not

Spec-Driven Development is not anti-AI. The methodology depends on AI — to construct code, to evaluate quality, to run automated checks that would be impractical for humans alone. AI is faster and more capable inside this process than outside it.

It is not bureaucracy. It does not add review meetings, sign-off chains, or documentation overhead for its own sake. The gates are automated where automation is sufficient and human where judgment is genuinely required.

It is not a replacement for skilled people. It is the process that makes skilled people more effective — because they spend their time on specification and judgment rather than on prompt iteration and rework.

What it is: **total visibility and control over what AI is building, why it is building it, and whether the result matches what the organisation asked for.** The same visibility that organisations have spent years trying to recover from the spreadsheet era — delivered by construction, not by retrospective governance effort.

Competitive Position

Spec-Driven Development changes where the competitive advantage in technology sits.

AI removes the scarcity in code construction. Building software is no longer the bottleneck. The bottleneck — and therefore the source of advantage — shifts to the quality of specification: the precision with which the business translates its requirements into something that can govern AI construction.

Teams with rigorous specification practice will build faster, with fewer findings, with cleaner audit trails, and with production systems that stay aligned to their specifications. They will deliver more features per delivery cycle, respond to change with lower operational risk, and withstand audit examination without reconstruction effort.

Teams without that practice will build faster than before, accumulate unverifiable behaviour faster than before, and face a growing inventory of systems that cannot be fully accounted for.

Genuine Agility

The most profound consequence of this model is not audit capability or governance compliance. It is agility in the original sense of the word — the actual ability to respond to change.

In conventional development, changing a system is expensive. Requirements must be re-gathered, designs revised, code rewritten, tests updated, deployments re-run. The cost of change increases as a system matures. Organisations learn to resist it.

When the specification is the source of truth and AI does the construction work, this relationship inverts. A regulatory change does not trigger a project. It triggers a specification update. The AI rebuilds what needs rebuilding against the new constraints, the same evaluators confirm conformance, and the audit trail records exactly what changed, why, and when.

A new market opportunity does not require assembling a team and planning a programme. It requires specifying what the new capability must do. The construction follows from the specification, at a speed that was previously impossible.

A single person with a clear specification and governed AI construction has the productive leverage that previously required a team. Not because the work is trivial — because the work is correctly divided: human judgment defines what the system must do, AI constructs it, and the methodology guarantees the result can be accounted for.

This is what genuine agility looks like: not faster sprints, but a fundamentally different relationship with change. The specification is the system. Change the specification, and the system follows.

The specification is not a technical document. It is not written by engineers and handed to developers. It is the organisation's knowledge of itself — what it does, what rules it operates under, what it must never get wrong. The WHAT. Expressed precisely enough

that a machine can build against it.

That precision is the competitive asset. Organisations that know themselves clearly enough to write it down can regenerate their systems on demand. Organisations that cannot will find that AI builds faster, but builds the wrong thing faster.

Current Status

This methodology is being developed using itself. One specification. Three independent AI development agents working against it simultaneously, each producing a distinct implementation. Each implementation is evaluated against the same formal criteria. The outputs are compared, gaps are identified, and the specification evolves accordingly.

The process is the proof. A methodology that cannot govern its own construction is not a governance methodology.

The formal specification, implementation, and architectural decision records are maintained at github.com/foolishimp/ai_sdlc_method.

Theoretical Foundations

The methodology described in this document is not a set of best practices assembled from experience. It is an application of a published formal theory to the problem of software delivery governance. The following papers establish the theoretical basis.

Constraint-Emergence Ontology: Reality as Self-Organising Constraint Network
Popov, D. (2026). Zenodo. https://zenodo.org/records/18682622

The foundational theory. Proposes that stable, governable structures — in physical systems, engineered systems, and organisations — emerge from constraint networks rather than from top-down design. The central claim relevant here: the pattern "encoded representation → constructor → constructed structure" recurs across substrates. Software development is one instance of this pattern. The specification is the encoded representation. The AI is the constructor. The delivered software is the constructed structure. Governance means governing the constraint network, not supervising the constructor.

Emergent Reasoning in Large Language Models: Soft Unification, Constraint Mechanisms, and Computational Traversal Popov, D. (2026). Zenodo. https://zenodo.org/records/18653552

The theoretical explanation for why AI models behave differently under formal constraints than under natural language prompts. The paper establishes that LLMs perform reasoning by traversing a learned structure, where hallucination — the production of plausible but incorrect output — occurs when constraints are sparse or

absent. This is the theoretical basis for the core claim of Spec-Driven Development: that a formal specification reduces the space within which the AI can produce non-conforming output. More constraints, fewer possible wrong answers.

Programming LLM Reasoning: A Meta-Template for Constraint Specifications
Popov, D. (2026). Zenodo. https://zenodo.org/records/18653642

The applied companion to the theory above. Describes the practical method for loading formal constraint specifications into an LLM such that it functions as an evaluator — testing whether outputs satisfy the specification — rather than as a generative system producing its best guess. The key distinction for practitioners: "The LLM does not execute the formal system — it predicts what execution would produce." This paper is the direct theoretical basis for the evaluator framework used in every stage gate described in this document.

The Political Operating System: Formal Constraint Specifications for Political Philosophy Popov, D. (2026). Zenodo. https://zenodo.org/records/18661906

Included as evidence that the constraint specification method generalises beyond software. The same technique — loading a formal specification into an LLM to produce governed, auditable, repeatable reasoning — has been applied to formalising political philosophies as executable systems. The significance for this document's audience: the method is not a software-development trick. It is a general approach to converting an AI from an opinion-generator into a rules-bound evaluator. Software delivery governance is one domain of application.

For Technical Leaders: What Changes at Each Stage

The table below shows what each stage of the delivery lifecycle looks like under three approaches: conventional development, AI prompt-driven development, and Spec-Driven Development. The question after each stage is the one that an auditor, a risk officer, or a senior technical reviewer would ask.

Requirements

	Traditional	Prompt-Driven	Spec-Driven
How it works	Workshops, Word documents, Jira tickets, informal sign-off	Developer describes what they want in natural language; AI interprets	Every requirement has a unique identifier (REQ-*) defined once in a formal document,

	Traditional	Prompt-Driven	Spec-Driven
			reviewed and baselined
Who verifies completeness	A human reviewer, informally	Nobody — the AI does not know what it doesn't know	An automated check confirms every requirement identifier is covered before delivery can advance
Audit answer: "What were you asked to build?"	Find the right version of the right document	Reconstruct from conversation history	Read the baselined requirements file; every item has a unique key and a version

---

Feature Decomposition and Planning

	Traditional	Prompt-Driven	Spec-Driven
How it works	Sprint planning, backlog grooming, estimation sessions	"Break this down into tasks" — AI suggests a plan	Requirements are formally decomposed into features; each feature declares which requirements it satisfies
Completeness check	Team judgment; gaps surface during build	No check — missing requirements are discovered late or not at all	Automated gate: every requirement identifier must be claimed by at least one feature. Delivery cannot proceed until the check passes
Audit answer: "Did you plan for everything?"	Difficult to prove without reconstructing meeting notes	Cannot prove	Show the coverage report: N requirements, all N covered, timestamp, approver

The gate here is not a review meeting. It is a rule the system enforces: if any requirement is uncovered, the system reports the gap and will not advance. A human then either adds the missing feature or disputes the requirement. Either way, the decision is recorded.

Architecture and Design

	Traditional	Prompt-Driven	Spec-Driven
How it works	Architects document decisions in Confluence or Visio; developers interpret	"Design this system" — AI proposes an architecture with no knowledge of your constraints	Technology choices, integration patterns, and constraints are encoded as formal Architectural Decision Records (ADRs). Each ADR traces to the requirements it satisfies
Technology constraints	Documented in a wiki; may or may not be followed	The AI uses whatever it finds plausible	The constraint surface is loaded into every subsequent AI invocation as mandatory context. The AI cannot ignore it
Audit answer: "Why was this architectural decision made?"	Find the relevant Confluence page and hope it is current	Cannot answer	The ADR is numbered, dated, states the alternatives considered, the trade-offs, and the requirements it addresses

The volume here matters. This project has over thirty architectural decision records, each governing a specific aspect of how the system behaves. That is the constraint surface. It is not a style guide or a set of suggestions — it is the formal definition of what conforming implementations must do.

Development

	Traditional	Prompt-Driven	Spec-Driven
How it works	Developers write code manually	AI generates code; developer reviews and iterates	AI generates code against the specification; automated and AI-

	Traditional	Prompt-Driven	Spec-Driven
			assisted evaluators check conformance before the output is accepted
Quality gate	Code review, manual testing	Developer judgment	The work cannot close until it passes a defined set of checks. A partial or non-conforming output does not advance
Audit answer: "Which requirement does this code satisfy?"	Read comments if they exist; ask the developer	Cannot reliably answer	Every code unit is tagged with the requirement identifier it implements. The tag is searchable and verifiable

---

Testing

	Traditional	Prompt-Driven	Spec-Driven
How it works	QA team writes and runs test cases, often manually	"Did you test this?" — AI reports it tested; coverage is informal	Test cases are generated and tagged to the requirement they validate. Coverage is a measurable number, not an estimate
Completeness	Depends on QA team capacity and judgment	Unknown	Every requirement identifier must appear in at least one test. The same automated check that governs feature decomposition applies here

	Traditional	Prompt-Driven	Spec-Driven
Audit answer: "How do you know this requirement is validated?"	Show the test case and hope the mapping is documented	Cannot answer	Show the test tagged REQ-F-AUTH-001 and the record showing it passed

---

Deployment

	Traditional	Prompt-Driven	Spec-Driven
How it works	Release process, change advisory board, manual verification	Build what the AI produced and deploy it	Every artifact deployed carries a fingerprint computed at the point it was accepted. The deployment record includes what was deployed, from which verified state, and which requirements it satisfies
Change record	Change management ticket, often completed after the fact	Informal	The delivery record is generated continuously during build. The deployment record is a projection of it, not a separate document
Audit answer: "What exactly was deployed on this date?"	Find the release notes and hope they are complete	Cannot answer	Read the delivery record: requirements satisfied, artifacts fingerprinted, evaluators that confirmed acceptance

---

Production Operations

	Traditional	Prompt-Driven	Spec-Driven
How it works	Monitoring, alerting, incident response	Wait for something to break, then re-prompt a fix	The production system emits signals tagged to the same requirement identifiers used in development. Deviation from specified behaviour is detected automatically
Drift detection	Humans notice when something breaks	No mechanism	When a signal tagged REQ-F-AUTH-001 shows anomalous behaviour, the system identifies which specified requirement is in drift and initiates a governed response
Audit answer: "How do you know the live system still satisfies this requirement?"	Manual review, penetration testing, periodic audit	Cannot answer	The production monitor is watching it continuously, using the same requirement identifier that governed its construction

---

Appendix: This Is Not a Prompt

A reasonable question from anyone familiar with AI tooling is: what makes Spec-Driven Development different from sophisticated prompt engineering? The answer is the nature of the documents that govern AI behaviour.

A prompt is a natural language instruction. It is interpreted by the model, which applies its best judgement about intent. Two prompts that appear similar can produce meaningfully different outputs. There is no formal definition of what a correct response looks like.

A constraint document is different in kind. It specifies rules that must be satisfied, stated precisely enough that a system can check compliance automatically. The model is not interpreting intent — it is working within a defined space and will be evaluated against defined criteria.

Below are two fragments from this project's constraint documents. There are currently over thirty such documents in the specification. Each governs a specific aspect of how any conforming implementation must behave.

Fragment 1: The Completeness Gate (ADR-S-013)

This document governs what it means for the feature planning stage to be complete. The following is the formal convergence rule — the condition that must be satisfied before delivery advances to architecture.

> **The feature decomposition stage converges when, and only when, two conditions are both true:**
>
> **Condition A — Coverage Check (automated):** Every requirement identifier defined in the requirements document must appear in at least one feature's declared scope. This is computed automatically. If any requirement is missing, the check fails. There is no partial credit.
>
> **Condition B — Human Approval:** A human reviewer must explicitly confirm that the feature list is the correct decomposition of the requirements — not merely complete, but correctly structured, correctly ordered, and correctly scoped to the agreed delivery boundary.
>
> *Coverage alone is not sufficient. A feature list can cover every requirement and still be the wrong plan — wrong granularity, wrong sequencing, wrong MVP boundary. The automated check gates the human review. The human review gates advancement.*

What is notable here is not just the rule — it is that this rule applies uniformly to every implementation, every team, every delivery. There is no version of this methodology where a project skips the coverage check because the team felt confident. The gate is not a guideline.

Fragment 2: The Execution Contract (ADR-S-015)

This document governs how work is recorded at runtime. Every unit of work — every stage transition — is treated as a transaction with the following structure.

> **Every stage transition opens a transaction when it begins and closes it only when work is confirmed complete. The closing record is the commit point.**

Phase	What is recorded	Meaning
Begin	START — input artifacts fingerprinted	Work has commenced; prior state is on record
Execute	Nothing	Work is in progress; no commitment yet
Accept	COMPLETE — output artifacts fingerprinted	Work is confirmed; this is the point of record
Reject	FAIL or ABORT	Prior state remains authoritative

> *An artifact written to the system without a corresponding COMPLETE record is uncommitted work. On restart, the system detects the open transaction, compares the current state of every affected file against the fingerprints recorded at the start, and flags any file that was modified but never committed. This is how the system detects crashes, partial writes, and incomplete AI outputs.*

The fingerprint referred to here is a SHA-256 hash — a standard cryptographic technique that produces a unique, verifiable signature for any file. If the file changes by a single character, the hash changes. This is not novel technology. What is novel is applying it systematically to every artifact at every stage of software delivery, so that the audit trail is mathematically verifiable rather than document-based.

These two fragments represent one layer of the constraint surface. The full specification includes rules governing how requirements are tagged, how architecture decisions are recorded, how tests map to requirements, how production signals link to the specifications they monitor, and how conflicts between requirements are resolved. Each rule is stated precisely, versioned, and applied uniformly.

That is the difference between a prompt and a specification.